

Theory Questions

Model Context Protocol (MCP) and FastMCP

MCP Fundamentals

1. Explain the difference between a tool, a resource, and a prompt in MCP, with a one-line example of when you'd reach for each.

A **tool** is something the model can call when it needs to do an action or get a result. A tool normally takes some input, performs some work, and returns a result. It can be useful for actions such as calculations, searching, or adding a new record. For example, I would use a tool to add a new expense like “Add Rs. 120 for lunch.”

A **resource** is data that is already available and can be read by the client as context. It is normally identified by a URI, so the client can ask for a particular piece of information without treating it as an action. For example, I would use a resource like `expenses://today` to show today's saved expenses.

A **prompt** is a reusable instruction template that the user can select for a specific task. It helps keep a common instruction ready instead of writing the same instruction again each time. For example, I would use a prompt like “Review today's spending and give me a short summary.”

The main difference is that a tool is used to **do something**, a resource is used to **provide information**, and a prompt is used to **guide the model with a reusable instruction**. In practice, the three can work together in one application, where a tool changes or calculates something, a resource provides the related data, and a prompt gives a ready-made way to use that information.

2. A local MCP server usually talks to Claude Desktop over stdio instead of a normal HTTP API. What does that mean in practice, and why does it make sense for this use case?

`stdio` means standard input and standard output. In a local MCP setup, Claude Desktop starts the MCP server as a local process and communicates with it through the process's input and output streams.

In simple terms, it works like this:

Claude Desktop <-> stdin/stdout <-> MCP Server

This makes sense for a local server because I do not need to create a separate HTTP service or open a network port. Claude can start the server when it needs it and communicate with it directly. It keeps the setup simple when both the client and server are running on the same computer.

FastMCP Documentation Questions

3. FastMCP supports three transports: stdio, http, and sse. Read the deployment docs and explain when you'd pick stdio vs http, and what actually changes in your `mcp.run()` call to switch between them.

I would choose `stdio` when the MCP server is mainly for local use, such as Claude Desktop, local testing, or a single-user application. It is the default transport, so I can simply write:

```
mcp.run()
```

I would choose `http` when I want the MCP server to be available through a network, used by more than one client, or deployed as a remote service. In FastMCP, this means using Streamable HTTP:

```
mcp.run(transport="http", host="127.0.0.1", port=8000)
```

For SSE, the code is similar:

```
mcp.run(transport="sse", host="127.0.0.1", port=8000)
```

SSE is mainly kept for older-client compatibility, while HTTP is the recommended choice for new network-based projects. So the main change is the `transport` value. With `stdio`, I normally do not pass a transport argument; for HTTP, I set `transport="http"` and normally give the host and port.

4. FastMCP automatically turns some tool return values into “structured output” and others into plain text. Read the tools docs and explain the difference between what happens when a tool returns a plain string vs when it returns a dict or a Pydantic model, and why that distinction would matter to a client calling your tool.

If a FastMCP tool returns a normal string, FastMCP sends it as normal text content. For example:

```
return "Expense added successfully"
```

This is mainly useful when the result just needs to be read as a message.

If the tool returns a dictionary or a Pydantic model, FastMCP can create structured output from the object. For example:

```
{  
  "amount": 120,  
  "category": "Food",  
  "total": 850  
}
```

Now the client can receive the individual fields such as `amount`, `category`, and `total`, instead of only getting one text sentence. FastMCP also keeps the normal content representation, so clients can still get readable content as well.

This matters because structured data is easier for a client to validate, read, and use in another program. A client does not have to parse values from a sentence.